Final Project

Secure Virtual Assistant with Speaker Recognition

1 Overview

- In recent years, virtual assistants have become increasingly popular and have been adopted across a wide range of domains. These systems can support users in performing various tasks, such as retrieving information, setting reminders, controlling devices, accessing personal data, and modifying system settings.
- However, not every function of a virtual assistant should be executed immediately without additional verification. For sensitive or important tasks, the system must authenticate the user before allowing the requested action to be performed. In addition, speaker identification can be used to personalize the user experience for appropriate functions.
- In this project, students will develop a virtual assistant that integrates speaker verification for important tasks and speaker identification for personalization. Specifically, students will select a speaker verification or speaker identification model, train and evaluate it using a suitable dataset, and integrate the model into a complete virtual assistant application.

2 Requirements

1. **(5 points)** Students must select a representative model for speaker verification or speaker identification, such as ECAPA-TDNN, RawNet3, or an equivalent model, and use a suitable dataset to train and evaluate it. The report must clearly describe the dataset, the train, validation, and test data splits, the selected model, the training procedure, appropriate evaluation metrics, and the experimental results.
2. **(5 points)** Using the model developed in Requirement 1, students must build a complete virtual assistant that integrates speaker verification and speaker identification. The system must be designed around specific use cases proposed by the students and satisfy the following requirements:
   - The virtual assistant must support voice-based interaction with users. The system must also provide a speaker enrollment and management component, such as a web interface or a simple application, for collecting initial voice data and managing the information required for speaker verification and identification.

- The system must include at least one general function that does not require authentication, at least one important function that may only be performed after successful speaker verification, and at least one function that uses speaker identification to personalize responses, information, or settings for each registered user.
- For the speaker verification and identification features, students must clearly describe the enrollment procedure for a new user, including how the initial voice data is collected and how the information required for speaker verification and identification is stored.
- The report must clearly present the overall system architecture and processing flow.

3 Suggested Virtual Assistant Implementation

- A typical virtual assistant architecture may be organized as a pipeline consisting of speech signal acquisition, Automatic Speech Recognition (ASR), request analysis and task orchestration, and Text-to-Speech (TTS) for generating spoken responses. For important tasks, the system must include a speaker verification (SV) step before executing the corresponding action. In addition, a speaker identification (SID) component may be incorporated to support personalization for appropriate functions.
- The request analysis and task orchestration module may be implemented using one of the following approaches:
  - **Rule-based:** Develop a set of rules and command patterns that directly map user inputs to corresponding tasks.
  - **Intent–Entity-based:** Formulate the problem as intent classification and entity extraction, followed by mapping the identified intent and entities to the corresponding action. Frameworks such as Rasa may be used to implement this approach.
  - **LLM-based:** Use Large Language Models (LLMs) to infer user intent and orchestrate the corresponding tasks.

4 Submission Guidelines

- Students must submit the complete source code, trained model weights, training dataset, and project report.
- All submission materials must be packaged into a single ZIP file. The filename must contain the student IDs of all team members, separated by underscores, using the following format:

`StudentID1_StudentID2_..._StudentIDN.zip`

- If the dataset or trained model is too large to include in the ZIP file, students may upload the materials to Google Drive and submit a text file containing the corresponding link. The text file must be named using the following format:

`StudentID1_StudentID2_..._StudentIDN.txt`